Restaurant Ordering System

Hearth & Table | Software Design Patterns | 13 November 2026

Team: Anuar, Akmaral and Mukhammad

GitHub: https://github.com/ANAYSSA/Final-Project-SDP

Website: https://hearth-table.178.62.244.88.sslip.io

1. Introduction

We developed a restaurant ordering application with Java 21 and Spring Boot. The interface uses HTML, CSS and JavaScript. Customers can choose pizza, pasta or drinks, add toppings, select dine-in or takeaway, apply a discount and place an order. The order page shows status updates and receipts. Prices are in KZT. The project continues the pizzeria examples from our earlier assignments.

2. Team contributions

Member	Work and Git evidence
Anuar (ANAYSSA)	Builder and Factory Method; initial application, web integration and deployment. Baseline: **954a3c8**.
Akmaral (arynttai)	FAMILY discount: **1debe3f**. JSON receipt renderer: **7475264**. Unit/API tests. Merged in PR #1.
Mukhammad (Vertex-blip)	Herbs decorator: **f51cc9c**. Operations observer: **b2e5b55**. Event display and tests. Merged in PR #2.

Both pull requests were merged without squashing. The original authors and implementation commits remain in the repository.

3. Patterns used

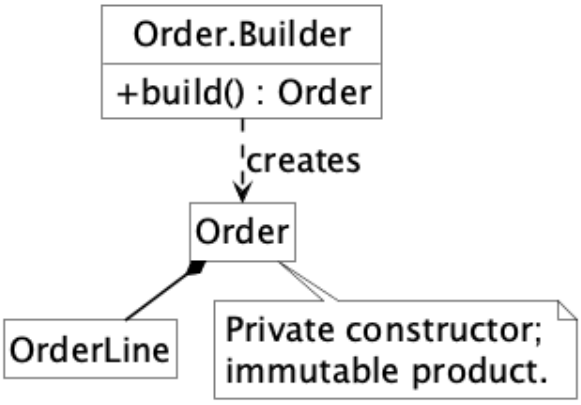
Pattern / type	Use in the application
Builder / creational	Order.Builder assembles and validates an immutable order. Dine-in requires a table; takeaway must not include one.
Factory Method / creational	MealFactory delegates creation to PizzaFactory, PastaFactory or DrinkFactory. Checkout works with the Meal interface.
Decorator / structural	Toppings wrap a Meal. Herbs adds 150 KZT per unit and can combine with other toppings. Drinks reject toppings.
Strategy / behavioral	PricingCalculator uses one DiscountStrategy. FAMILY gives 15% off a decorated subtotal of at least 6000 KZT. Lower subtotals are rejected.
Bridge / structural	CustomerReceipt and KitchenTicket use the same renderer contract. Each supports text, HTML or JSON without separate classes for every combination.
Observer / behavioral	OrderProgress notifies customer, kitchen and operations subscribers. Operations messages contain only a short reference, status and item quantity.

The source has six pattern packages with role subfolders. Main is the only class at the package root. HTTP adapters are separate from the domain classes. Quote and order creation share the same calculation, and BigDecimal handles money with scale 2 and HALF_UP rounding.

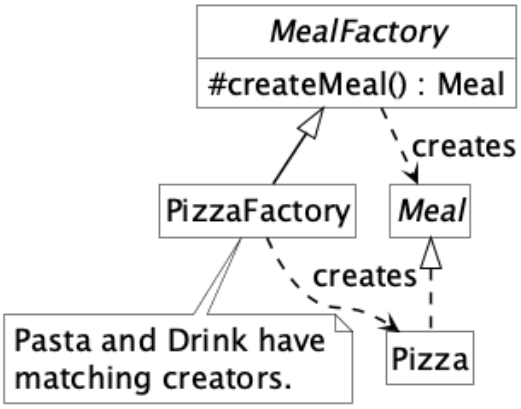
4. UML class diagrams

These diagrams show the main roles. Full diagrams and editable PlantUML sources are included in docs/uml. Some concrete variants are omitted here to keep the diagrams readable.

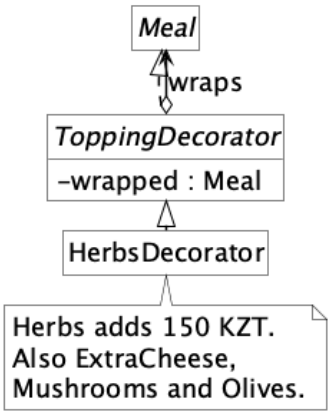
Builder



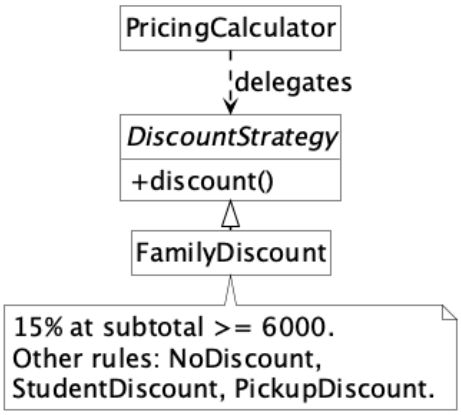
Factory Method



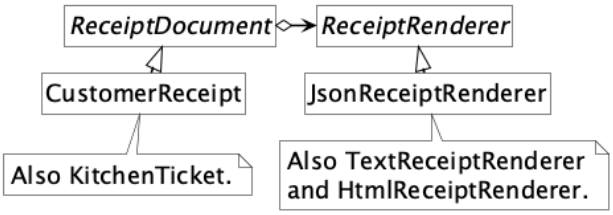
Decorator



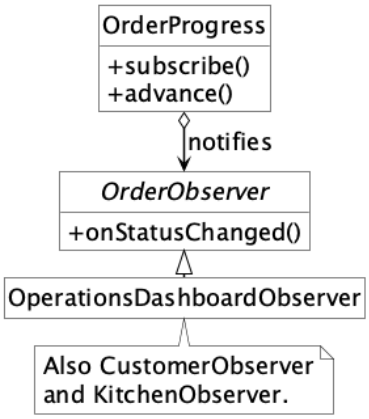
Strategy



Bridge



Observer



Factory creates the meal; decorators add extras; Builder creates the order; Strategy calculates its price; Observer publishes progress; Bridge produces the selected receipt.

5. Code examples and verification

Selected excerpts from the Java implementation; complete classes and tests are in the repository.

Builder - OrderService.java

```
Order.Builder builder = new Order.Builder()
    .customer(request.customerName()).fulfillment(request.fulfillment())
    .tableNumber(request.tableNumber()).note(request.note())
    .discount(discounts.find(request.discountCode()));
buildLines(request.items()).forEach(builder::addItem);
Order order = builder.build();
```

Factory Method - MealFactory.java

```
public final Meal create(MenuDefinition definition) {
    Objects.requireNonNull(definition, "A menu definition is required");
    if (definition.category() != category)
        throw new IllegalArgumentException("Wrong meal category for factory");
    return createMeal(definition);
}
protected abstract Meal createMeal(MenuDefinition definition);
```

Decorator - HerbsDecorator.java

```
public final class HerbsDecorator extends ToppingDecorator {
    public HerbsDecorator(Meal meal) { super(meal, ToppingType.HERBS); }
}
```

Strategy - FamilyDiscount.java

```
public BigDecimal discount(BigDecimal subtotal, Fulfillment fulfillment) {
    if (subtotal.compareTo(MINIMUM_SUBTOTAL) < 0)
        throw new IllegalArgumentException(
            "Family discount requires a subtotal of at least " + MINIMUM_SUBTOTAL
            + " KZT; this order is " + subtotal + " KZT");
    return subtotal.multiply(RATE);
}
```

Bridge - JsonReceiptRenderer.java

```
public String render(ReceiptContent content) {
    ObjectNode receipt = JSON.createObjectNode();
    receipt.put("title", content.title());
    content.lines().forEach(receipt.putArray("lines")::add);
    try {
        return JSON.writeValueAsString(receipt);
    } catch (JsonProcessingException exception) {
        throw new UncheckedIOException("Receipt could not be encoded as JSON", exception);
    }
}
```

Observer - OperationsDashboardObserver.java

```
@Override public OrderNotification onStatusChanged(StatusChange change) {
    String reference = change.orderId().toString().substring(0, 8).toUpperCase(Locale.ROOT);
    String message = "Order " + reference + " | " + change.status() + " | Items: " + itemQuantity;
    return new OrderNotification(change.status(), message, "OPERATIONS", change.occurredAt());
}
```

Verification

The combined project passed **88 tests with no failures or errors** on Java 21. Tests cover discount boundaries, topping composition, JSON/HTML escaping, status transitions, concurrent retries and access from another session. The console --demo and package-structure check also pass.

Example checked in the browser: three Margheritas with herbs cost 6450 KZT. FAMILY subtracts 967.50 KZT, giving a total of 5482.50 KZT. Advancing the status adds customer, kitchen and operations events.

Conclusion and further work

All six patterns work in one ordering flow. Narrow interfaces keep pricing, rendering and notification responsibilities separate; shared calculations avoid duplicating pricing rules. Orders currently live in the browser session and disappear after expiry or server restart. The next steps would be a database and staff authentication, followed by a kitchen screen to replace the simulated status button.

6. Application screenshots

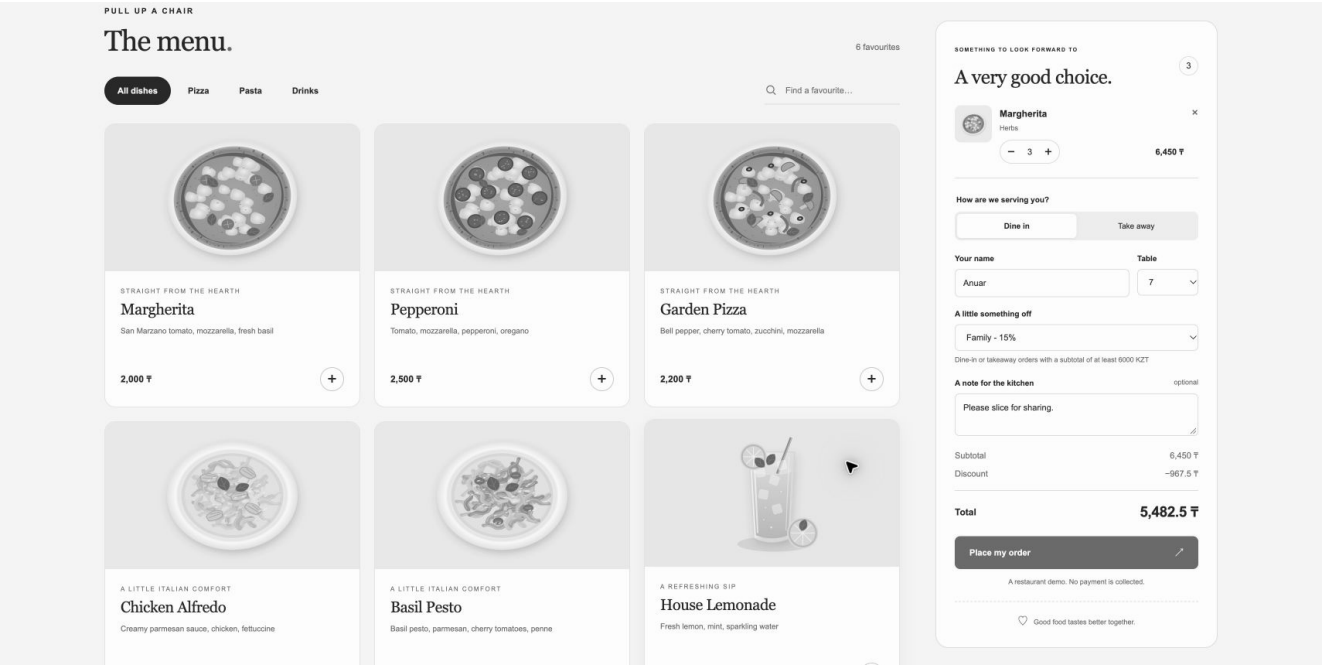


Figure 1. The deployed menu and checkout with the merged Herbs topping and FAMILY discount.

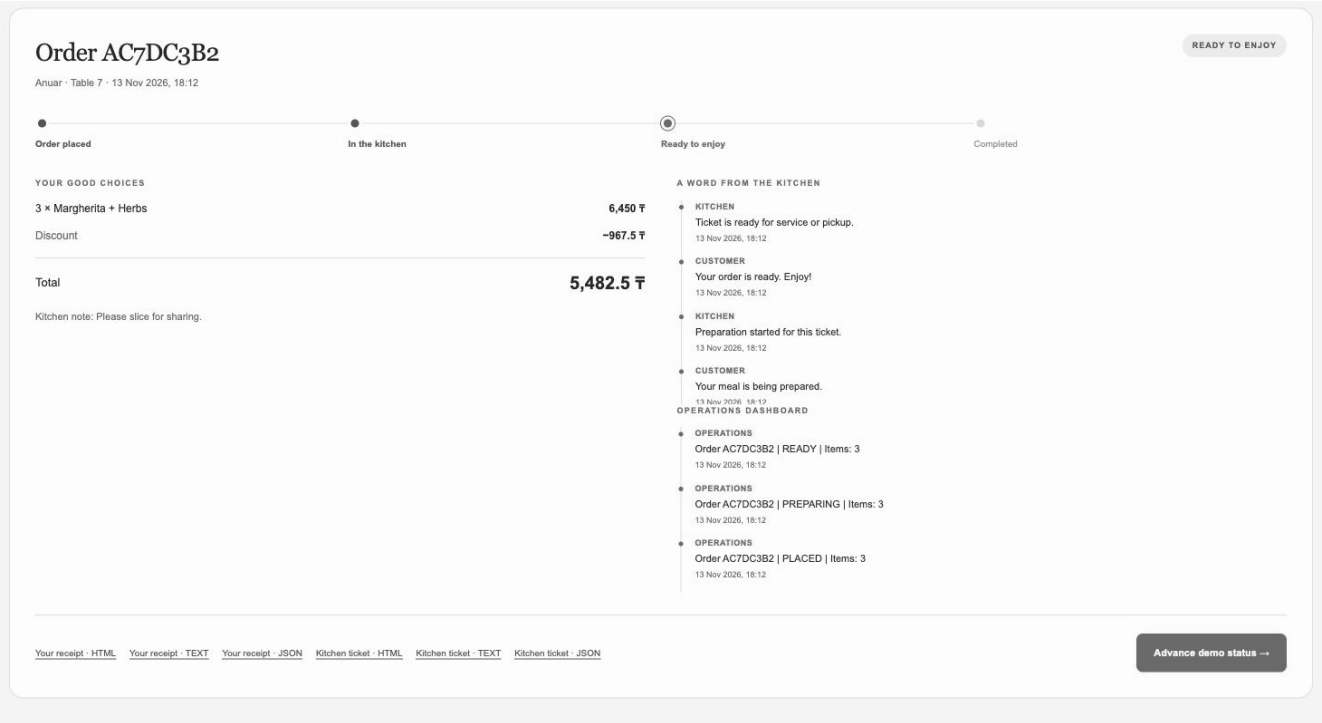


Figure 2. Order progress, session-scoped notifications and receipt links in text, HTML and JSON.

The application runs on DigitalOcean behind an HTTPS reverse proxy. This is a course demonstration: it does not collect payments or send orders to a real restaurant. The status button simulates preparation.